SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	GANESHWAR S	**Reg. No.:**	192421342
Date:	03.08.2026	**Duration:**	60 minutes

Code of Conduct

I Ganeshwar S / 192421342 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Bheeshma L

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design	Produces a clear, complete, and well-structured architecture diagram with all	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and	Diagram is incomplete, unclear, or

	major components, interfaces, and interactions accurately represented.		lacks sufficient detail.	technically incorrect.
Component Interaction and Quality Attribute Analysis	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis	/5
Selection of Software Architecture	/5
Architecture Diagram and Component Design	/5
Component Interaction and Quality Attribute Analysis	/5
Architectural Justification and Technical Presentation	/5
TOTAL	/5

Table of Contents

1. System Requirement Analysis

1.1 Problem Recap and System Objectives

AgriSmart is a precision-agriculture platform unifying IoT-based field monitoring, AI-driven crop advisory, task and inventory management, and a direct-to-buyer marketplace for farmers, farm supervisors, agronomists, and buyers. The system objectives that most strongly influence architectural choices are: near real-time ingestion and processing of high-volume IoT sensor data; independently evolvable AI advisory capability (crop recommendation and image-based pest/disease diagnosis); reliable transaction processing for the marketplace and payments; and the ability to scale to many farms, regions, and crop types without service-wide downtime.

1.2 Functional Requirements Influencing Architecture

- Continuous ingestion and real-time display of IoT sensor data (soil moisture, temperature, humidity, pH) from potentially thousands of field devices.
- Automated and manual irrigation scheduling that reacts to sensor and weather data.
- AI-based crop recommendation and image-based pest/disease diagnosis, each with distinct compute profiles (lightweight rule engine vs. GPU-backed inference).
- Task assignment/tracking and inventory management for farm labour.
- A marketplace supporting produce listings, order placement, and secure payment processing.
- Role-based authentication across five distinct actor types with different access levels.

1.3 Non-Functional Requirements Influencing Architecture

Quality Attribute	Architectural Implication
Scalability	Sensor ingestion and marketplace traffic will grow independently and unevenly across farms/seasons, requiring components that scale in isolation rather than as one monolith.
Maintainability	AI models (crop advisory, pest diagnosis) will be retrained and redeployed frequently and independently of core transactional logic.
Reliability	Irrigation alerts and payment transactions are safety/financial-critical and must tolerate partial failures without cascading.
Availability	Field monitoring and alerting must remain available close to 24x7, even during maintenance of unrelated modules such as the marketplace.
Performance	Sensor data must be processed and surfaced within seconds; AI inference may take longer and must not block other real-time flows.
Security	Financial transactions and personal farm data require strong authentication, encryption, and isolation between services.

Taken together, these quality attributes point toward an architecture that decomposes the system into independently deployable, independently scalable units, communicating through both synchronous APIs (for direct user-facing requests) and asynchronous events (for high-volume sensor streams and cross-service notifications).

2. Selection of Software Architecture

2.1 Selected Architectural Style

A Hybrid Architecture combining Microservices with an Event-Driven backbone, fronted by an API Gateway, is selected for AgriSmart. Client-facing requests (registration, task assignment, marketplace browsing) are handled through synchronous REST APIs via the gateway, while high-volume, high-frequency data — IoT sensor readings, alerts, order/inventory events — flows through an asynchronous event bus (Kafka/MQTT).

2.2 Justification

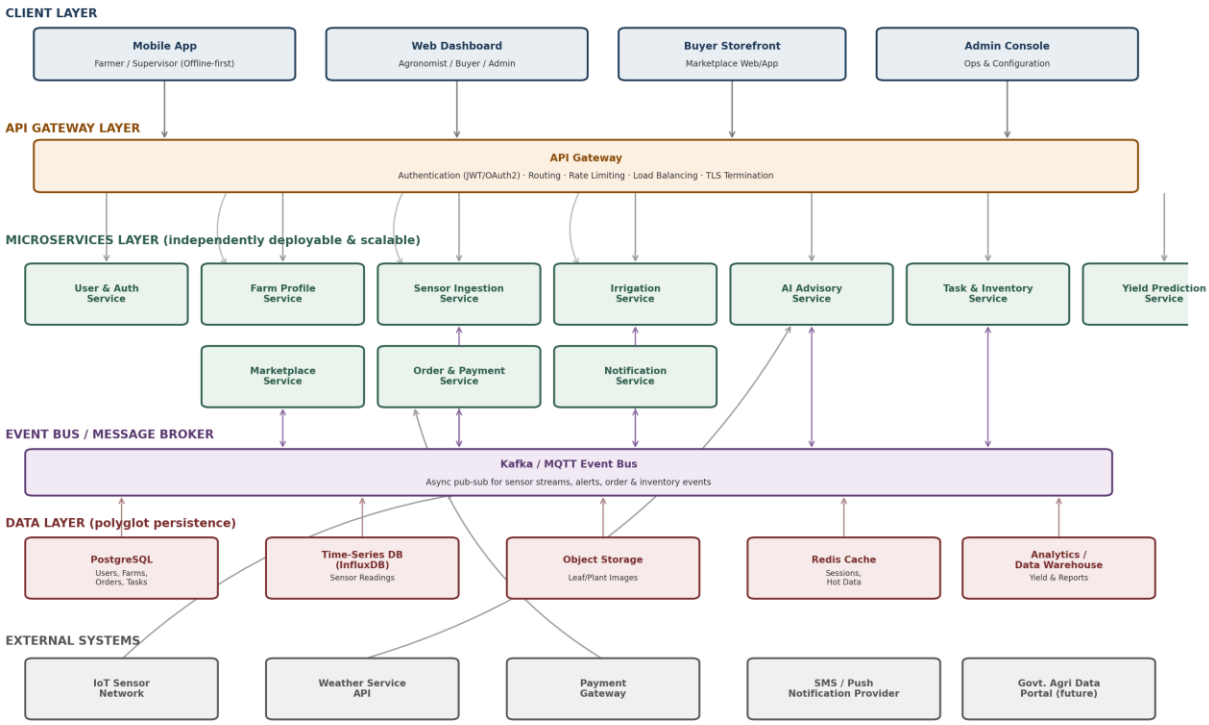
- Independent Scalability: The Sensor Ingestion Service and AI Advisory Service can be scaled horizontally during peak growing season without over-provisioning the Marketplace or User services, which follow different load patterns.
- Technology Heterogeneity: A microservices boundary allows the AI Advisory Service to use Python/ML frameworks and GPU-backed inference, while transactional services (Orders, Payments) use a conventional relational-database stack — a single monolith would force one technology choice for all concerns.
- Fault Isolation: If the AI Advisory Service is degraded or under heavy load, irrigation alerts and marketplace transactions continue to function, since they are deployed and scaled as separate services.
- Event-Driven Backbone for IoT: Sensor data arrives continuously and asynchronously from thousands of field devices; a synchronous request-response model would not handle this volume or intermittent connectivity gracefully, whereas a publish-subscribe event bus naturally buffers and fans out this data to multiple consumers (dashboards, alerting, AI advisory, analytics).
- Cross-Disciplinary Alignment: Decoupling the AI/agronomy-related services from the core transactional services mirrors the cross-disciplinary nature of the problem domain — agronomy/data-science concerns evolve on a different cadence than standard business/CRUD concerns.

Alternative styles were considered and rejected as primary choices: a pure Layered/Monolithic architecture would simplify initial development but would not allow the AI Advisory and Sensor Ingestion components to scale or fail independently of the rest of the system. A pure Serverless (FaaS) approach was considered for sensor ingestion specifically but was not adopted platform-wide due to the need for long-running, stateful AI inference and predictable latency for irrigation alerts; it remains a candidate for specific bursty functions (e.g., image pre-processing) as a future refinement.

3. Software Architecture Diagram

The diagram below shows the complete AgriSmart architecture: the client layer, API Gateway, the microservices layer, the event bus, the polyglot data layer, and external systems, along with the synchronous (REST) and asynchronous (event) communication paths between them.

AgriSmart – Hybrid Microservices + Event-Driven Architecture



Legend: → synchronous request/response (REST) ↔ asynchronous event (pub/sub)

Cross-cutting concerns (applied across all layers): Service Discovery & Load Balancing, Centralized Logging & Monitoring, Container Orchestration (Kubernetes), CI/CD Pipeline, Distributed Tracing.

4. Components and Their Interactions

4.1 Component Responsibilities

Component	Layer	Responsibility
Mobile App / Web Dashboard / Buyer Storefront / Admin Console	Client Layer	Role-specific front-ends for farmers, supervisors, agronomists, buyers, and administrators; the mobile app supports offline-first data entry for low-connectivity field use.
API Gateway	Gateway Layer	Single entry point for all client requests; handles authentication (JWT/OAuth2), request routing to the correct microservice, rate limiting, load balancing, and TLS termination.
User & Auth Service	Microservice	Manages user accounts, roles, and credentials; issues and validates access tokens used by the gateway and other services.
Farm Profile Service	Microservice	Manages farm/field registration, location, crop type, and area metadata.
Sensor Ingestion Service	Microservice	Receives streaming data from the IoT sensor network (via MQTT), validates and timestamps readings, and publishes them onto the event bus for downstream consumption.
Irrigation Service	Microservice	Consumes sensor and weather events to generate irrigation schedules and threshold-based alerts.
AI Advisory Service	Microservice	Runs crop recommendation and image-based pest/disease diagnosis models; consumes sensor/weather data and image uploads, publishes diagnostic results and recommendations.
Task & Inventory Service	Microservice	Manages field-task assignment/tracking and inventory levels for supervisors and labourers.
Yield Prediction Service	Microservice	Aggregates historical and current-season data from the analytics warehouse to generate yield forecasts.
Marketplace Service	Microservice	Manages produce listings, search, and availability for buyers/distributors.
Order & Payment Service	Microservice	Handles order placement, coordinates with the external Payment Gateway, and publishes order/payment events.
Notification Service	Microservice	Subscribes to alert, task, and order events and dispatches push/SMS notifications to the relevant users.

Component	Layer	Responsibility
Kafka/MQTT Event Bus	Middleware	Asynchronous publish-subscribe backbone carrying sensor readings, alerts, and order/inventory events between producer and consumer services, decoupling them in time and load.
Data Layer (PostgreSQL, Time-Series DB, Object Storage, Redis, Data Warehouse)	Persistence	Polyglot persistence: relational data (users, farms, orders) in PostgreSQL, high-frequency sensor readings in a time-series database, images in object storage, hot session/cache data in Redis, and historical data for analytics in a data warehouse.

4.2 Interaction and Workflow Example

A typical end-to-end flow illustrates how components interact: (1) an IoT sensor reports low soil moisture, which the Sensor Ingestion Service validates and publishes to the event bus; (2) the Irrigation Service consumes this event, checks it against the field's threshold and the latest weather forecast (fetched via the Weather API), and decides to trigger irrigation; (3) it publishes an 'irrigation-triggered' event and an 'alert' event; (4) the Notification Service consumes the alert event and pushes a notification to the farmer's mobile app; (5) all events and resulting readings are simultaneously persisted to the time-series database and data warehouse for the Yield Prediction Service to use later. Client-facing actions, such as a buyer placing an order, instead flow synchronously: Buyer Storefront to API Gateway to Marketplace Service to Order & Payment Service to the external Payment Gateway, with the resulting order event then published to the bus so the Notification Service can inform the farmer.

5. Quality Attribute Analysis

5.1 Scalability

Each microservice can be scaled independently and horizontally (e.g., via Kubernetes replica sets); the Sensor Ingestion and AI Advisory services — which face the highest and most variable load — can be scaled up during peak growing season without affecting the Marketplace or Admin services. The event bus itself scales by adding broker partitions/nodes, allowing sensor throughput to grow with the number of onboarded farms.

5.2 Security

The API Gateway centralizes authentication and enforces TLS for all client traffic; internal service-to-service calls run within a private network/service mesh with mutual TLS. Role-based access control is enforced by the User & Auth Service, and the Order & Payment Service isolates cardholder-related processing to the certified external Payment Gateway rather than storing sensitive payment data itself, reducing PCI-DSS scope.

5.3 Performance

The event-driven design ensures the high-frequency sensor stream never blocks user-facing requests, since ingestion and dashboard queries are handled by separate services. Redis caching is used for frequently accessed data (e.g., current field status, marketplace listings) to keep response times low, while the more latency-tolerant AI Advisory inference runs asynchronously and returns results via notification rather than blocking the request path.

5.4 Reliability

The event bus provides durable, replayable message storage, so if the Irrigation Service or Notification Service is briefly unavailable, sensor events are not lost and are processed once the service recovers. Each microservice owns its own datastore, preventing a failure or slow query in one service (e.g., Yield Prediction) from directly degrading another (e.g., Order & Payment).

5.5 Maintainability

Clear service boundaries aligned to business capability (farm profile, irrigation, AI advisory, marketplace, etc.) mean that each service can be developed, tested, and deployed by a small team independently, and the AI Advisory Service's models can be retrained and redeployed without touching transactional services. Consistent API contracts at the gateway and event schemas on the bus reduce the risk of breaking changes.

5.6 Availability

Because services are deployed independently across multiple container instances with load balancing and health checks, the failure of a single instance does not take down the whole platform. Critical services — Sensor Ingestion, Irrigation, and Notification — can be given higher replica counts and priority scheduling to help meet the 99.5% uptime target defined for core monitoring and alerting in the SRS.

6. Justification of Architectural Decisions and Trade-offs

6.1 Rationale Summary

The hybrid microservices + event-driven style was chosen because AgriSmart's workload is genuinely heterogeneous: constant low-latency IoT ingestion, occasional heavy AI inference, and standard CRUD/transactional marketplace operations. No single classical style (pure layered, pure client-server, pure SOA) addresses all three workload types equally well, whereas decomposing along these lines and connecting them with an event bus lets each part be built, scaled, and evolved with the technology and cadence appropriate to it.

6.2 Trade-offs Considered

- Operational Complexity vs. Flexibility: Microservices introduce deployment, monitoring, and network complexity compared to a monolith; this is accepted in exchange for independent scalability and fault isolation, and mitigated with container orchestration and centralized logging/tracing.

- Eventual Consistency vs. Simplicity: Using an event bus means some data (e.g., a newly placed order reflected in inventory) is eventually, not immediately, consistent across services; this is acceptable for AgriSmart's domain, where a few seconds of propagation delay does not affect correctness of farm operations, in exchange for much higher ingestion throughput and resilience.
- Cost vs. Isolation: Running separate datastores per service (polyglot persistence) costs more in infrastructure and operational overhead than one shared database, but is justified by the very different data shapes and access patterns of sensor time-series data, transactional records, and images.

6.3 Support for Future Enhancements and Long-Term Maintainability

- New capabilities (e.g., drone imagery analysis, autonomous machinery integration, government subsidy portal integration) can be added as new microservices that subscribe to existing events without modifying existing services.
- The API Gateway provides a stable integration point for future client applications (e.g., a dedicated agronomist tablet app) without requiring changes to backend services.
- The event-driven backbone allows the platform to expand from a single pilot region to many regions by adding partitions/consumer groups, supporting the scalability goals defined in the CO2 outcome for this assessment.
- Because services are loosely coupled through well-defined APIs and event schemas, individual components (e.g., replacing the AI Advisory model, or swapping the payment gateway provider) can be upgraded or replaced with minimal impact on the rest of the system, directly supporting long-term maintainability.

7. Conclusion

This architecture design report has analyzed the requirements and quality attributes driving AgriSmart's design, selected and justified a Hybrid Microservices + Event-Driven architecture fronted by an API Gateway, presented a complete architecture diagram covering all major components and external systems, explained component responsibilities and interactions through a representative workflow, evaluated the design against the six key quality attributes, and discussed the trade-offs and future-enhancement rationale behind the chosen design. The resulting architecture is scalable, resilient, and maintainable, and is well aligned with the cross-disciplinary nature of a precision-agriculture platform